MODULE 8 – ASP.NET CORE & REST API DEVELOPMENT

1. Introduction to ASP.NET Core

ASP.NET Core is a modern, open-source, cross-platform web development framework developed by Microsoft.

It is used to build:

- Web applications.
- REST APIs.
- MVC applications.
- Real-time applications.
- Cloud-based applications.
- Microservices.

Features of ASP.NET Core:

1. Cross-platform support.
2. High performance.
3. Built-in Dependency Injection.
4. Middleware-based request pipeline.
5. Authentication and authorization support.
6. Configuration and logging support.
7. Integration with Entity Framework Core.
8. Supports REST API development.
9. Supports asynchronous programming.
10. Open-source framework.

2. What is a Web API?

API stands for Application Programming Interface.

An API is a mechanism that allows two software applications to communicate with each other.

A Web API exposes functionality and data over a network, commonly using HTTP.

Real-world example:

Suppose you are developing a Student Management System.

The frontend application needs to display student information.

Instead of directly accessing the database, the frontend sends an HTTP request to the ASP.NET Core Web API.

The API processes the request, communicates with the database and returns the response.

Basic Communication Flow:

```
None
Frontend Application
        |
        v
HTTP Request
        |
        v
ASP.NET Core Web API
        |
        v
Business Logic
        |
        v
Entity Framework Core
        |
        v
SQL Server Database
        |
        v
HTTP Response
        |
        v
Frontend Application
```

3. What is REST?

REST stands for Representational State Transfer.

REST is an architectural style used to design network-based applications and APIs.

REST APIs commonly use HTTP methods to perform operations on resources.

Important REST Principles:

1. Client-Server Architecture: Client and server have separate responsibilities.
2. Statelessness: Each request contains the information required to process it. The server does not depend on stored client session state to understand the request.
3. Cacheability: Responses should indicate whether they can be cached.
4. Uniform Interface: Resources are accessed using a consistent interface.
5. Layered System: The client does not need to know every component between itself and the server.
6. Code on Demand: An optional constraint that allows executable code to be sent to clients.

4. What is a REST API?

A REST API is an API designed according to REST architectural principles.

It uses HTTP requests and responses to communicate with resources.

Example resource:

```
None
/api/students
```

Common operations:

HTTP Method	Purpose
GET	Retrieve data
POST	Create a new resource
PUT	Replace or update a resource
PATCH	Partially update a resource
DELETE	Delete a resource

Example URLs:

```
None
GET     /api/students
GET     /api/students/1
```

```
POST    /api/students
PUT     /api/students/1
PATCH   /api/students/1
DELETE  /api/students/1
```

5. HTTP Protocol

HTTP stands for Hypertext Transfer Protocol.

It is an application-layer protocol used for communication between clients and servers.

A client sends an HTTP request, and the server returns an HTTP response.

HTTP Request Contains:

- HTTP method.
- URL.
- Headers.
- Optional body.

HTTP Response Contains:

- Status code.
- Headers.
- Optional response body.

Example HTTP request:

```
None
GET /api/students/1 HTTP/1.1
Host: localhost:7057
Accept: application/json
```

Example HTTP response:

```
None
HTTP/1.1 200 OK
Content-Type: application/json
```

```
{
  "id": 1,
  "name": "Divya",
  "age": 21
}
```

6. HTTP Methods

HTTP methods describe the operation the client wants to perform.

1. GET

Used to retrieve data from the server.

Example:

```
None
GET /api/students
```

2. POST

Used to create a new resource.

Example:

```
None
POST /api/students
```

3. PUT

Used to replace an existing resource or create it when the API supports that behavior.

Example:

```
None
PUT /api/students/1
```

4. PATCH

Used to partially update a resource.

Example:

```
None
PATCH /api/students/1
```

5. DELETE

Used to delete a resource.

Example:

```
None
DELETE /api/students/1
```

7. Idempotency in HTTP

An operation is idempotent when performing it multiple times has the same intended effect on the server state as performing it once.

Examples:

- GET: Idempotent.
- PUT: Idempotent by HTTP semantics.
- DELETE: Idempotent by HTTP semantics.
- POST: Not generally idempotent.
- PATCH: Depends on the operation being performed.

Example:

Sending the same DELETE request multiple times should leave the resource deleted after the first successful deletion, even though later responses may be different.

8. HTTP Status Codes

Status codes indicate the result of an HTTP request.

1xx – Informational

Indicates that the request is being processed.

2xx – Success

- 200 OK: Request completed successfully.
- 201 Created: A new resource was created.
- 202 Accepted: Request accepted for processing.
- 204 No Content: Request succeeded, but no response body is returned.

3xx – Redirection

- 301 Moved Permanently.
- 302 Found.
- 304 Not Modified.

4xx – Client Errors

- 400 Bad Request: Invalid request.
- 401 Unauthorized: Authentication is required or has failed.
- 403 Forbidden: Access is not permitted.
- 404 Not Found: Resource not found.
- 405 Method Not Allowed: HTTP method is not supported for the requested resource.
- 409 Conflict: Request conflicts with the current state of the resource.
- 415 Unsupported Media Type: The request content type is not supported.
- 422 Unprocessable Content: Request content cannot be processed according to the application's rules.
- 429 Too Many Requests: Too many requests were sent.

5xx – Server Errors

- 500 Internal Server Error.
- 501 Not Implemented.
- 502 Bad Gateway.
- 503 Service Unavailable.
- 504 Gateway Timeout.

9. Creating an ASP.NET Core Web API Project

A Web API project can be created using Visual Studio or the .NET CLI.

Using .NET CLI:

```
None
dotnet new webapi -n StudentManagementAPI
```

Navigate to the project:

```
None
cd StudentManagementAPI
```

Open the project:

```
None
code .
```

Run the project:

```
None
dotnet run
```

Using Visual Studio:

1. Open Visual Studio.
2. Select Create a new project.
3. Select ASP.NET Core Web API.
4. Enter the project name.
5. Select the required .NET version.
6. Configure authentication if required.
7. Create the project.

10. ASP.NET Core Web API Project Structure

A typical Web API project contains:

```
None
StudentManagementAPI/
│
├── Controllers/
│   └── StudentsController.cs
│
├── Models/
│   └── Student.cs
│
├── DTOs/
│   ├── StudentRequestDto.cs
```

```
|   └── StudentResponseDto.cs
|
├── Data/
|   └── AppDbContext.cs
|
├── Services/
|   ├── IStudentService.cs
|   └── StudentService.cs
|
├── Repositories/
|   ├── IStudentRepository.cs
|   └── StudentRepository.cs
|
├── Middleware/
|
├── Program.cs
├── appsettings.json
└── StudentManagementAPI.csproj
```

Explanation:

- Controllers: Handles HTTP requests and responses.
- Models: Contains application and entity models.
- DTOs: Defines data transferred between client and API.
- Data: Contains database-related classes.
- Services: Contains business logic.
- Repositories: Handles data-access operations.
- Middleware: Contains custom request pipeline components.
- Program.cs: Application entry point and configuration.
- appsettings.json: Stores application settings.

11. Program.cs

Program.cs is the entry point of a modern ASP.NET Core application.

It configures application services and the HTTP request pipeline.

Example:

```
None
var builder = WebApplication.CreateBuilder(args);

// Register services
builder.Services.AddControllers();

builder.Services.AddEndpointsApiExplorer();

builder.Services.AddSwaggerGen();

var app = builder.Build();

// Configure middleware
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();

app.UseAuthorization();

app.MapControllers();

app.Run();
```

Explanation:

- WebApplication.CreateBuilder(): Creates the application builder.
- builder.Services: Registers application services.
- AddControllers(): Registers controller-related services.
- AddEndpointsApiExplorer(): Registers endpoint metadata exploration.
- AddSwaggerGen(): Registers Swagger generation when the corresponding package is installed.
- builder.Build(): Creates the WebApplication instance.
- UseSwagger(): Enables Swagger JSON generation.
- UseSwaggerUI(): Enables interactive Swagger UI.
- UseHttpsRedirection(): Redirects HTTP requests to HTTPS where configured.

- UseAuthorization(): Enables authorization middleware.
- MapControllers(): Maps controller routes.
- Run(): Starts the application.

12. What is a Controller?

A controller is a class that handles incoming HTTP requests and returns HTTP responses.

In ASP.NET Core Web API, controllers commonly inherit from ControllerBase.

Example:

```
None
using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/[controller]")]
public class StudentsController : ControllerBase
{
    [HttpGet]
    public IActionResult GetStudents()
    {
        return Ok("Students retrieved successfully");
    }
}
```

Explanation:

- [ApiController]: Enables API-specific behaviors.
- [Route("api/[controller]")]: Defines the route pattern.
- StudentsController: Controller class.
- ControllerBase: Provides API-related controller functionality.
- [HttpGet]: Maps the action to HTTP GET requests.
- IActionResult: Represents an HTTP action result.
- Ok(): Returns HTTP 200 OK.

13. Controller vs ControllerBase

ControllerBase:

- Provides functionality for handling Web API requests.
- Supports action results such as Ok(), NotFound() and BadRequest().

- Does not provide MVC view functionality.

Controller:

- Inherits from ControllerBase.
- Provides additional functionality for MVC applications.
- Supports returning views using View().

Which should we use?

For Web API controllers, we generally use ControllerBase because APIs return data and HTTP responses rather than MVC views.

For MVC controllers that return views, we use Controller.

14. What is the [ApiController] Attribute?

[ApiController] is an attribute used to enable API-specific behaviors on a controller.

Important features:

1. Automatic model validation responses.
2. Binding-source inference.
3. Improved error responses.
4. Requires attribute routing for API controllers.

Example:

```
None
[ApiController]
[Route("api/[controller]")]
public class StudentsController : ControllerBase
{
}
```

When model validation fails, ASP.NET Core can automatically return HTTP 400 Bad Request without requiring manual ModelState checks in the action.

15. Routing in ASP.NET Core Web API

Routing determines which controller action handles an incoming HTTP request.

Example:

```
None
[Route("api/[controller]")]
public class StudentsController : ControllerBase
{
    [HttpGet]
    public IActionResult GetAll()
    {
        return Ok();
    }

    [HttpGet("{id}")]
    public IActionResult GetById(int id)
    {
        return Ok(id);
    }
}
```

Routes:

```
None
GET /api/students
GET /api/students/1
```

Attribute Routing:

Routes are defined using attributes.

```
None
[HttpGet("active")]
public IActionResult GetActiveStudents()
{
    return Ok();
}
```

Route:

```
None
GET /api/students/active
```

Conventional Routing:

Routes are defined centrally using route patterns. It is commonly used in MVC applications.

16. HTTP Attributes in Controllers

ASP.NET Core provides attributes to map controller actions to HTTP methods.

- [HttpGet]: Handles GET requests.
- [HttpPost]: Handles POST requests.
- [HttpPut]: Handles PUT requests.
- [HttpPatch]: Handles PATCH requests.
- [HttpDelete]: Handles DELETE requests.

Example:

```
None
[ApiController]
[Route("api/[controller]")]
public class StudentsController : ControllerBase
{
    [HttpGet]
    public IActionResult GetAll()
    {
        return Ok();
    }

    [HttpPost]
    public IActionResult Create()
    {
        return Ok();
    }

    [HttpPut("{id}")]
    public IActionResult Update(int id)
    {
```

```
        return Ok();
    }

    [HttpDelete("{id}")]
    public IActionResult Delete(int id)
    {
        return NoContent();
    }
}
```

17. IActionResult and ActionResult<T>

These types represent responses returned by controller actions.

IActionResult:

Used when an action may return different HTTP response types.

Example:

```
None
[HttpGet("{id}")]
public IActionResult GetStudent(int id)
{
    if (id <= 0)
    {
        return BadRequest();
    }

    return Ok("Student found");
}
```

ActionResult<T>:

Used when an action returns a particular data type or an HTTP action result.

Example:

```
None
[HttpGet("{id}")]
public ActionResult<Student> GetStudent(int id)
{
    var student = new Student
    {
        Id = id,
        Name = "Divya"
    };

    return Ok(student);
}
```

Difference:

- IActionResult: Returns different kinds of action results.
- ActionResult<T>: Can return a typed result or another action result.

18. HTTP Helper Methods in ControllerBase

ASP.NET Core provides helper methods to return appropriate HTTP responses.

Important methods:

```
None
return Ok(data);
return CreatedAtAction(nameof(GetById), new { id = student.Id },
student);
return NoContent();
return BadRequest("Invalid request");
return NotFound("Student not found");
return Conflict("Duplicate record");
return Unauthorized();
return Forbid();
return StatusCode(500, "Internal Server Error");
```

Explanation:

- Ok(): Returns 200.

- CreatedAtAction(): Returns 201 and can provide the location of the created resource.
- NoContent(): Returns 204.
- BadRequest(): Returns 400.
- NotFound(): Returns 404.
- Conflict(): Returns 409.
- Unauthorized(): Returns 401.
- Forbid(): Returns 403.
- StatusCode(): Returns a specified status code.

19. Model Binding

Model Binding is the process of converting incoming HTTP request data into C# parameters or model objects.

ASP.NET Core can bind data from:

- Route values.
- Query strings.
- Request body.
- Form fields.
- Headers.

Example:

```
None
[HttpGet("{id}")]
public IActionResult GetStudent(int id)
{
    return Ok(id);
}
```

For this request:

```
None
GET /api/students/10
```

The route value 10 is bound to the id parameter.

20. Model Binding Attributes

1. [FromRoute]

Gets a value from the route.

```
None
[HttpGet("{id}")]
public IActionResult GetStudent(
    [FromRoute] int id)
{
    return Ok(id);
}
```

Request:

```
None
GET /api/students/10
```

2. [FromQuery]

Gets a value from the query string.

```
None
[HttpGet("search")]
public IActionResult Search(
    [FromQuery] string name)
{
    return Ok(name);
}
```

Request:

```
None
GET /api/students/search?name=Divya
```

3. [FromBody]

Gets data from the request body.

```
None
[HttpPost]
public IActionResult Create(
    [FromBody] Student student)
{
    return Ok(student);
}
```

4. [FromHeader]

Gets data from an HTTP header.

```
None
[HttpGet("details")]
public IActionResult GetDetails(
    [FromHeader(Name = "X-Client-Id")] string clientId)
{
    return Ok(clientId);
}
```

5. [FromForm]

Gets values from form fields.

```
None
[HttpPost("upload")]
public IActionResult Upload(
    [FromForm] string description)
{
    return Ok(description);
}
```

21. Model in ASP.NET Core

A model represents data used by an application.

Example:

```
None
public class Student
{
    public int Id { get; set; }

    public string Name { get; set; } = string.Empty;

    public int Age { get; set; }

    public string Department { get; set; } = string.Empty;
}
```

Models can represent:

- Database entities.
- Application data.
- Request and response structures.

22. Data Transfer Objects (DTOs)

DTO stands for Data Transfer Object.

A DTO is an object used to transfer data between different parts of an application or between the client and server.

Why use DTOs?

1. Avoid exposing unnecessary entity properties.
2. Control request and response data.
3. Improve security.
4. Separate API contracts from database entities.
5. Support validation.
6. Avoid allowing clients to modify protected fields.

Example entity:

```
None
public class Student
{
    public int Id { get; set; }
```

```
    public string Name { get; set; } = string.Empty;

    public int Age { get; set; }

    public string Department { get; set; } = string.Empty;

    public DateTime CreatedAt { get; set; }
}
```

Request DTO:

```
None
public class StudentRequestDto
{
    public string Name { get; set; } = string.Empty;

    public int Age { get; set; }

    public string Department { get; set; } = string.Empty;
}
```

Response DTO:

```
None
public class StudentResponseDto
{
    public int Id { get; set; }

    public string Name { get; set; } = string.Empty;

    public string Department { get; set; } = string.Empty;
}
```

Here, the client cannot directly provide the database-generated Id or CreatedAt through the request DTO.

23. Data Validation

Validation ensures that incoming data meets the required rules before processing.

ASP.NET Core supports validation using attributes and other validation approaches.

Common validation attributes:

- [Required]
- [StringLength]
- [MinLength]
- [MaxLength]
- [Range]
- [EmailAddress]
- [RegularExpression]
- [Phone]

Example:

```
None
using System.ComponentModel.DataAnnotations;

public class StudentRequestDto
{
    [Required]
    [StringLength(100)]
    public string Name { get; set; } = string.Empty;

    [Range(18, 60)]
    public int Age { get; set; }

    [Required]
    public string Department { get; set; } = string.Empty;
}
```

With [ApiController], invalid model states automatically produce a 400 response by default.

24. What is Middleware?

Middleware is a component that is part of the ASP.NET Core HTTP request pipeline.

Each middleware component can:

- Process an incoming request.
- Call the next middleware.
- Process the outgoing response.
- Stop further processing by not calling the next middleware.

Example:

```
None
app.Use(async (context, next) =>
{
    Console.WriteLine("Before Middleware");

    await next();

    Console.WriteLine("After Middleware");
});
```

25. Middleware Pipeline

The middleware pipeline defines the order in which HTTP requests are processed.

Example:

```
None
HTTP Request
     |
     v
Exception Handling
     |
     v
HTTPS Redirection
     |
     v
Static Files (if required)
     |
     v
Routing
     |
     v
```

```
CORS (if required)
     |
     v
Authentication
     |
     v
Authorization
     |
     v
Controller Endpoint
     |
     v
HTTP Response
```

Middleware execution order is important.

For example:

- Authentication should run before authorization.
- Routing should run before endpoint-specific authorization when using the conventional endpoint pipeline.
- CORS should be placed appropriately, usually after routing and before authorization when using endpoint routing.

26. Built-in Middleware

ASP.NET Core provides built-in middleware.

Examples:

- UseHttpsRedirection(): Redirects HTTP to HTTPS.
- UseStaticFiles(): Serves static files.
- UseRouting(): Enables routing.
- UseCors(): Enables Cross-Origin Resource Sharing.
- UseAuthentication(): Identifies the user.
- UseAuthorization(): Checks access permissions.
- UseExceptionHandler(): Handles application exceptions.
- UseResponseCaching(): Enables configured response caching.

27. Custom Middleware

Custom middleware allows developers to implement their own request-processing logic.

Common uses:

- Request logging.
- Exception handling.
- Request tracking.
- Custom security checks.
- Performance monitoring.

Example:

```
None
public class RequestLoggingMiddleware
{
    private readonly RequestDelegate _next;

    private readonly ILogger<RequestLoggingMiddleware> _logger;

    public RequestLoggingMiddleware(
        RequestDelegate next,
        ILogger<RequestLoggingMiddleware> logger)
    {
        _next = next;
        _logger = logger;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        _logger.LogInformation(
            "Request: {Method} {Path}",
            context.Request.Method,
            context.Request.Path);

        await _next(context);
    }
}
```

Registering middleware:

```
None
app.UseMiddleware<RequestLoggingMiddleware>();
```

28. Dependency Injection in ASP.NET Core

Dependency Injection is a design pattern used to provide the required objects to a class instead of creating them manually.

ASP.NET Core has built-in Dependency Injection.

Service Registration:

```
None
builder.Services.AddTransient<IStudentService, StudentService>();

builder.Services.AddScoped<IStudentRepository,
StudentRepository>();

builder.Services.AddSingleton<IApplicationInfo, ApplicationInfo>();
```

Three Service Lifetimes:

1. Transient:
    - A new instance is created each time the service is requested from the container.
2. Scoped:
    - One instance is created per scope.
    - In a typical Web API request, it is one instance per HTTP request.
3. Singleton:
    - One instance is shared for the lifetime of the service provider.

Constructor Injection:

```
None
public class StudentsController : ControllerBase
{
    private readonly IStudentService _studentService;

    public StudentsController(IStudentService studentService)
    {
```

```
        _studentService = studentService;
    }
}
```

The DI container provides the StudentService implementation through the constructor.

29. Service Layer

The service layer contains the business logic of an application.

It separates business rules from controller and database responsibilities.

Example interface:

```
None
public interface IStudentService
{
    Task<IEnumerable<StudentResponseDto>> GetAllAsync();
}
```

Implementation:

```
None
public class StudentService : IStudentService
{
    public async Task<IEnumerable<StudentResponseDto>>
GetAllAsync()
    {
        // Business logic and data access
        return await Task.FromResult(
            new List<StudentResponseDto>());
    }
}
```

Registration:

```
None
builder.Services.AddScoped<IStudentService, StudentService>();
```

30. Repository Pattern

The Repository Pattern is a design pattern that separates data-access logic from business logic.

A repository provides methods to perform operations on data.

Example interface:

```
None
public interface IStudentRepository
{
    Task<Student?> GetByIdAsync(int id);

    Task<List<Student>> GetAllAsync();

    Task AddAsync(Student student);

    void Update(Student student);

    void Delete(Student student);
}
```

A repository implementation uses DbContext to communicate with the database.

Advantages:

- Separates data-access logic.
- Provides a consistent data-access interface.
- Can simplify unit testing.
- Helps organize larger applications.

Important:
EF Core's DbContext and DbSet already provide repository-like and unit-of-work functionality.
An additional repository layer is an architectural choice, not a requirement for every project.

31. Entity Framework Core Integration

Entity Framework Core is an Object-Relational Mapper (ORM).

It allows developers to interact with relational databases using C# objects.

Important Concepts:

DbContext:

- Manages database operations.
- Tracks entity changes.
- Saves changes to the database.

DbSet:

- Represents a collection of entities of a particular type.
- Provides access to entity queries and operations.

DbContextOptions:

- Provides database configuration such as the provider and connection string.

Example DbContext:

```
None
public class AppDbContext : DbContext
{
    public AppDbContext(
        DbContextOptions<AppDbContext> options)
        : base(options)
    {
    }

    public DbSet<Student> Students { get; set; }
}
```

32. Database Connection Configuration

The database connection string is commonly stored in appsettings.json.

Example:

```
None
{
  "ConnectionStrings": {
```

```
    "DefaultConnection":
"Server=.\\SQLEXPRESS;Database=StudentDB;Trusted_Connection=True;Tr
ustServerCertificate=True;"
  }
}
```

Registering DbContext:

```
None
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(

builder.Configuration.GetConnectionString("DefaultConnection")
    )
);
```

This configures EF Core to use SQL Server.

The actual connection string must match the installed SQL Server instance and authentication configuration.

33. CRUD Operations in Web API

CRUD stands for:

- Create.
- Read.
- Update.
- Delete.

These are the basic operations performed on database records.

1. CREATE – POST

Adds a new student.

```
None
[HttpPost]
public async Task<IActionResult> Create(StudentRequestDto dto)
```

```
{
    var student = new Student
    {
        Name = dto.Name,
        Age = dto.Age,
        Department = dto.Department
    };

    _context.Students.Add(student);

    await _context.SaveChangesAsync();

    return CreatedAtAction(
        nameof(GetById),
        new { id = student.Id },
        student);
}
```

2. READ – GET

Retrieves all students.

```
None
[HttpGet]
public async Task<IActionResult> GetAll()
{
    var students = await _context.Students.ToListAsync();

    return Ok(students);
}
```

Retrieves a specific student.

```
None
[HttpGet("{id}")]
public async Task<IActionResult> GetById(int id)
```

```
{
    var student = await _context.Students.FindAsync(id);

    if (student == null)
    {
        return NotFound();
    }

    return Ok(student);
}
```

3. UPDATE – PUT

Updates an existing student.

```
None
[HttpPut("{id}")]
public async Task<IActionResult> Update(
    int id,
    StudentRequestDto dto)
{
    var student = await _context.Students.FindAsync(id);

    if (student == null)
    {
        return NotFound();
    }

    student.Name = dto.Name;
    student.Age = dto.Age;
    student.Department = dto.Department;

    await _context.SaveChangesAsync();

    return NoContent();
}
```

4. DELETE – DELETE

Deletes an existing student.

```
None
[HttpDelete("{id}")]
public async Task<IActionResult> Delete(int id)
{
    var student = await _context.Students.FindAsync(id);

    if (student == null)
    {
        return NotFound();
    }

    _context.Students.Remove(student);

    await _context.SaveChangesAsync();

    return NoContent();
}
```

34. Asynchronous Programming in Web API

Asynchronous programming allows applications to perform operations without blocking the calling thread while waiting for I/O operations.

It is especially useful for:

- Database queries.
- External API calls.
- File operations.
- Network communication.

Important keywords and types:

- async: Declares an asynchronous method.
- await: Awaits an asynchronous operation.
- Task: Represents an asynchronous operation without a result.
- Task<T>: Represents an asynchronous operation that returns a value.

Example:

```
None
[HttpGet]
public async Task<IActionResult> GetStudents()
{
    var students = await _context.Students.ToListAsync();

    return Ok(students);
}
```

Advantages:

- Improves server scalability for I/O-bound operations.
- Avoids blocking a request thread while waiting for I/O.
- Helps an application handle concurrent requests more efficiently.

35. Authentication

Authentication is the process of verifying the identity of a user.

It answers the question:

Who are you?

Examples:

- Username and password.
- JWT bearer token.
- Cookie authentication.
- External identity provider.

Example:

A user enters login details.

The application verifies the credentials and identifies the user.

36. Authorization

Authorization determines what an authenticated user is allowed to access.

It answers the question:

What are you allowed to do?

Example:

A Student Management System has two roles:

- Admin.
- Student.

Admin:

- Add students.
- Update students.
- Delete students.
- View all records.

Student:

- View permitted student information.
- Access their own profile.

37. Authentication vs Authorization

Authentication:

- Verifies user identity.
- Determines who the user is.
- Usually occurs before authorization.

Authorization:

- Checks access permissions.
- Determines what the user can do.
- Uses identities, claims, roles and policies.

38. JWT Authentication

JWT stands for JSON Web Token.

JWT is a compact token format commonly used to transmit claims between parties.

It is widely used in token-based authentication for Web APIs.

A JWT commonly contains three parts:

```
None
Header.Payload.Signature
```

1. Header

Contains information about the token type and signing algorithm.

2. Payload

Contains claims such as user identifier, role and expiration time.

3. Signature

Helps verify that the token has not been modified and was signed using the expected key or secret.

39. JWT Authentication Flow

1. User sends username and password to the login endpoint.
2. Server validates the credentials.
3. Server creates a signed JWT.
4. Server returns the token to the client.
5. Client stores the token securely.
6. Client sends the token with future requests.
7. API validates the token.
8. The authenticated user's claims become available to the application.
9. Authorization policies determine whether access is allowed.

Example request header:

```
None
Authorization: Bearer eyJhbGciOi...
```

Important:
A JWT payload is normally encoded, not encrypted. Do not place passwords, secrets or other sensitive information inside its claims.

40. Configuring JWT Authentication

Authentication configuration is commonly registered in Program.cs using the JWT bearer authentication handler.

Example:

```
None
using Microsoft.AspNetCore.Authentication.JwtBearer;
```

```
using Microsoft.IdentityModel.Tokens;
using System.Text;

builder.Services.AddAuthentication(
    JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters =
            new TokenValidationParameters
            {
                ValidateIssuer = true,
                ValidateAudience = true,
                ValidateLifetime = true,
                ValidateIssuerSigningKey = true,

                ValidIssuer = builder.Configuration["Jwt:Issuer"],
                ValidAudience =
builder.Configuration["Jwt:Audience"],

                IssuerSigningKey = new SymmetricSecurityKey(
                    Encoding.UTF8.GetBytes(
                        builder.Configuration["Jwt:Key"]!)
                )
            };
    });
```

Middleware configuration:

```
None
app.UseAuthentication();

app.UseAuthorization();
```

Authentication must run before authorization.

Secret keys should be stored securely using an appropriate secret-management mechanism rather than committed to source control.

41. Authorization Attributes

ASP.NET Core provides attributes to control access to endpoints.

[Authorize]

Requires authentication by default.

```
None
[Authorize]
[HttpGet]
public IActionResult GetProfile()
{
    return Ok("Authenticated user");
}
```

[AllowAnonymous]

Allows access without authentication.

```
None
[AllowAnonymous]
[HttpPost("login")]
public IActionResult Login()
{
    return Ok();
}
```

Role-Based Authorization

Restricts access to users who have the required role.

```
None
[Authorize(Roles = "Admin")]
[HttpDelete("{id}")]
public IActionResult DeleteStudent(int id)
{
    return NoContent();
}
```

Multiple roles can be specified:

```
None
[Authorize(Roles = "Admin,Manager")]
```

Here, a user with either role can satisfy the role requirement.

42. Policy-Based Authorization

Policy-based authorization allows developers to define reusable authorization rules.

Registering a policy:

```
None
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("AdminOnly", policy =>
        policy.RequireRole("Admin"));
});
```

Using the policy:

```
None
[Authorize(Policy = "AdminOnly")]
[HttpGet("admin")]
public IActionResult GetAdminData()
{
    return Ok("Admin data");
}
```

Policies can also use claims and custom authorization requirements.

43. Cross-Origin Resource Sharing (CORS)

CORS stands for Cross-Origin Resource Sharing.

It is a browser security mechanism that controls whether a web application from one origin can access resources from another origin.

Example:

- Frontend: https://localhost:4200
- Backend API: https://localhost:7057

Because the origins are different, the browser may enforce CORS restrictions.

Configure CORS:

```
None
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowFrontend", policy =>
    {
        policy.WithOrigins("https://localhost:4200")
              .AllowAnyHeader()
              .AllowAnyMethod();
    });
});
```

Use the policy:

```
None
app.UseRouting();

app.UseCors("AllowFrontend");

app.UseAuthentication();

app.UseAuthorization();

app.MapControllers();
```

Important:

- CORS is enforced by browsers.
- It is not an authentication mechanism.
- Use specific allowed origins in production.
- Avoid combining AllowAnyOrigin with AllowCredentials.

44. Swagger and OpenAPI

OpenAPI is a specification used to describe HTTP APIs.

Swagger is a collection of tools used to work with OpenAPI descriptions.

Swagger UI provides an interactive interface for exploring and testing API endpoints.

Advantages:

1. Displays API endpoints.
2. Shows request and response models.
3. Documents HTTP methods.
4. Shows response status codes.
5. Allows developers to send test requests.
6. Helps frontend and backend developers collaborate.

A common development setup uses Swagger UI to test API endpoints without requiring a separate frontend application.

45. Exception Handling in Web API

Exception handling is used to manage unexpected runtime errors.

ASP.NET Core provides built-in exception handling middleware.

Example:

```
None
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/error");
}
```

A more advanced implementation can use a dedicated exception handler and ProblemDetails responses.

Why use Global Exception Handling?

- Centralizes error handling.
- Avoids repeated try-catch blocks.
- Provides consistent error responses.
- Helps log unexpected failures.
- Prevents accidental exposure of internal exception details.

Example response:

```
None
{
  "title": "Internal Server Error",
  "status": 500,
  "detail": "An unexpected error occurred."
}
```

Internal stack traces and sensitive details should not be exposed to clients in production.

46. Logging in ASP.NET Core

Logging records application events and helps developers monitor and debug applications.

Example:

```
None
public class StudentService
{
    private readonly ILogger<StudentService> _logger;

    public StudentService(ILogger<StudentService> logger)
    {
        _logger = logger;
    }

    public void GetStudents()
    {
        _logger.LogInformation("Retrieving students");

        _logger.LogWarning("Student details are incomplete");

        _logger.LogError("Failed to retrieve students");
    }
}
```

Important logging levels:

- Trace.
- Debug.

- Information.
- Warning.
- Error.
- Critical.

47. API Versioning

API versioning allows developers to maintain multiple versions of an API.

It is useful when existing clients depend on an earlier API contract.

Example:

```
None
/api/v1/students
/api/v2/students
```

Reasons for versioning:

- Add new features.
- Change request or response structures.
- Maintain backward compatibility.
- Support older clients.
- Introduce new business requirements gradually.

Common versioning approaches:

- URL path versioning.
- Query string versioning.
- HTTP header versioning.
- Media type versioning.

48. Pagination

Pagination divides large result sets into smaller pages.

It avoids returning every database record in a single response.

Example:

```
None
[HttpGet]
public async Task<IActionResult> GetStudents(
```

```
    int pageNumber = 1,
    int pageSize = 10)
{
    if (pageNumber < 1 || pageSize < 1 || pageSize > 100)
    {
        return BadRequest("Invalid pagination parameters");
    }

    var students = await _context.Students
        .OrderBy(s => s.Id)
        .Skip((pageNumber - 1) * pageSize)
        .Take(pageSize)
        .ToListAsync();

    return Ok(students);
}
```

Explanation:

- Skip(): Skips a specified number of records.
- Take(): Retrieves the required number of records.
- OrderBy(): Ensures predictable ordering.

Example request:

```
None
GET /api/students?pageNumber=2&pageSize=10
```

49. Filtering and Sorting

Filtering retrieves records that satisfy specified conditions.

Example:

```
None
[HttpGet("search")]
public async Task<IActionResult> SearchStudents(
```

```
    string? department)
{
    var query = _context.Students.AsQueryable();

    if (!string.IsNullOrWhiteSpace(department))
    {
        query = query.Where(
            s => s.Department == department);
    }

    var students = await query.ToListAsync();

    return Ok(students);
}
```

Sorting arranges the records.

Example:

```
None
var students = await _context.Students
    .OrderBy(s => s.Name)
    .ToListAsync();
```

Descending order:

```
None
var students = await _context.Students
    .OrderByDescending(s => s.Name)
    .ToListAsync();
```

50. File Upload in ASP.NET Core

ASP.NET Core supports uploading files using IFormFile.

Example:

None

```
[HttpPost("upload")]
public async Task<IActionResult> UploadFile(
    IFormFile file)
{
    if (file == null || file.Length == 0)
    {
        return BadRequest("Please select a file");
    }

    var uploadsFolder = Path.Combine(
        Directory.GetCurrentDirectory(),
        "Uploads");

    Directory.CreateDirectory(uploadsFolder);

    var fileName = Path.GetRandomFileName();

    var filePath = Path.Combine(
        uploadsFolder,
        fileName);

    await using var stream = new FileStream(
        filePath,
        FileMode.Create);

    await file.CopyToAsync(stream);

    return Ok("File uploaded successfully");
}
```

Important file upload security practices:

- Validate file size.
- Validate permitted file types.
- Do not trust the original filename.
- Generate a safe server-side filename.
- Store uploaded files outside executable application folders where appropriate.
- Scan files when required.

- Avoid allowing uploaded files to execute on the server.

51. API Security Best Practices

Important security practices:

1. Use HTTPS.
2. Validate incoming data.
3. Use authentication where required.
4. Apply authorization policies.
5. Protect sensitive configuration values.
6. Use parameterized SQL queries.
7. Avoid exposing internal exception details.
8. Configure CORS carefully.
9. Apply rate limiting where appropriate.
10. Validate uploaded files.
11. Store passwords using secure password hashing.
12. Follow the principle of least privilege.
13. Keep dependencies updated.
14. Do not log passwords, tokens or sensitive user data.
15. Use appropriate token expiration and validation.

52. Rate Limiting

Rate limiting controls how many requests a client can send during a specified period.

It helps protect APIs from excessive traffic and resource abuse.

Example registration:

```
None
builder.Services.AddRateLimiter(options =>
{
    options.AddFixedWindowLimiter("fixed", limiter =>
    {
        limiter.PermitLimit = 10;
        limiter.Window = TimeSpan.FromMinutes(1);
    });
});
```

Apply middleware:

```
None
app.UseRateLimiter();
```

Apply the policy to an endpoint:

```
None
[EnableRateLimiting("fixed")]
[HttpGet]
public IActionResult GetStudents()
{
    return Ok();
}
```

A corresponding rate-limiting attribute namespace may be required:

```
None
using Microsoft.AspNetCore.RateLimiting;
```

The example limits the selected endpoint to ten permitted requests per one-minute window under that policy.

53. Health Checks

Health checks help determine whether an application or its dependencies are functioning.

They are useful for monitoring deployed applications.

Registration:

```
None
builder.Services.AddHealthChecks();
```

Mapping the endpoint:

```
None
app.MapHealthChecks("/health");
```

Request:

```
None
GET /health
```

Health checks can be extended to test dependencies such as databases and external services.

54. Response Caching

Response caching stores eligible HTTP responses so they may be reused.

It can reduce unnecessary processing and improve performance.

Example:

```
None
builder.Services.AddResponseCaching();
```

Middleware:

```
None
app.UseResponseCaching();
```

Controller action:

```
None
[HttpGet]
[ResponseCache(Duration = 60)]
public IActionResult GetPublicData()
{
    return Ok("Cached response");
}
```

Important:
Caching depends on response headers and cache rules. Do not cache sensitive user-specific responses unless the caching policy is designed securely.

55. Minimal APIs

Minimal APIs allow developers to create HTTP endpoints with less code and without creating controller classes.

Example:

```
None
var builder = WebApplication.CreateBuilder(args);

var app = builder.Build();

app.MapGet("/api/students", () =>
{
    return Results.Ok("All students");
});

app.MapGet("/api/students/{id}", (int id) =>
{
    return Results.Ok($"Student ID: {id}");
});

app.Run();
```

Advantages:

- Less boilerplate code.
- Simple endpoint creation.
- Suitable for small APIs and microservices.
- Supports Dependency Injection and middleware.

56. Controllers vs Minimal APIs

Controller-Based APIs:

- Organize endpoints using controller classes.
- Support attributes and action methods.
- Suitable for larger APIs with structured controllers.
- Provide familiar MVC-style organization.

Minimal APIs:

- Define endpoints directly using route-mapping methods.
- Require less boilerplate.
- Suitable for smaller services and lightweight endpoints.
- Support filters, authorization and dependency injection.

Both approaches can be used to build ASP.NET Core HTTP APIs.

57. Unit Testing Web API

Unit testing verifies individual units of application logic.

For example, a StudentService method can be tested independently.

Common .NET testing frameworks:

- xUnit.
- NUnit.
- MSTest.

Example:

```
None
public class Calculator
{
    public int Add(int a, int b)
    {
        return a + b;
    }
}
```

Test:

```
None
public class CalculatorTests
{
    [Fact]
    public void Add_ReturnsCorrectResult()
    {
        Calculator calculator = new Calculator();

        int result = calculator.Add(10, 20);

        Assert.Equal(30, result);
    }
}
```

58. Integration Testing Web API

Integration testing verifies how multiple application components work together.

Example:

- HTTP endpoint.
- Controller.
- Service.
- Database access.

ASP.NET Core provides WebApplicationFactory through its testing packages to host an application in tests and send HTTP requests to it.

Integration testing can verify:

- Endpoint responses.
- HTTP status codes.
- Request validation.
- Authentication behavior.
- Authorization behavior.
- Database interactions.

59. Deployment of ASP.NET Core Web API

Deployment makes an application available in a target environment.

Common deployment platforms:

- IIS.
- Microsoft Azure.
- Docker containers.
- Linux servers.
- Cloud hosting platforms.

Publish Command:

```
None
dotnet publish -c Release
```

Publishing prepares the application for deployment.

Important Deployment Steps:

1. Build the application.

2. Run tests.
3. Configure production connection strings.
4. Protect secrets.
5. Publish the application.
6. Configure the hosting environment.
7. Configure HTTPS.
8. Apply database migrations through a controlled deployment process.
9. Monitor application logs and health.
10. Verify deployed endpoints.

60. Complete Web API Architecture

A common layered ASP.NET Core Web API architecture looks like this:

```
None
Client Application
       |
       v
HTTP Request
       |
       v
Middleware Pipeline
       |
       v
Authentication
       |
       v
Authorization
       |
       v
Controller
       |
       v
DTO Validation
       |
       v
Service Layer
       |
       v
```

```
Repository Layer
       |
       v
DbContext
       |
       v
DbSet
       |
       v
SQL Server Database
       |
       v
HTTP Response
       |
       v
Client Application
```

Explanation:

- Client: Sends HTTP requests.
- Middleware: Processes requests and responses.
- Authentication: Identifies the user.
- Authorization: Checks permissions.
- Controller: Handles the HTTP request.
- DTO: Defines the request and response structure.
- Service: Executes business logic.
- Repository: Performs data-access operations when that layer is used.
- DbContext: Manages EF Core database operations.
- DbSet: Provides access to entity records.
- SQL Server: Stores application data.